

Intro to Data Science - HW 3

Copyright Jeffrey Stanton, Jeffrey Saltz, and Jasmina Tacheva

IST 687

Kent Roller

Homework 3

Due Date: 02/02/2024

Submission Date: 02/02/2024

```
# Enter your name here: Kent Roller
```

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

Reminders of things to practice from last week:

Make a data frame `data.frame()` Row index of `max()`/`min()` which `max()` which `min()` Sort value or order rows `sort()` `order()` Descriptive statistics `mean()` `sum()` `max()` Conditional statement `if (condition) "true stuff"` else `"false stuff"`

This Week:

Often, when you get a dataset, it is not in the format you want. You can (and should) use code to refine the dataset to become more useful. As Chapter 6 of Introduction to Data Science mentions, this is called “data munging.” In this homework, you will read in a dataset from the web and work on it (in a data frame) to improve its usefulness.

Part 1: Use `read_csv()` to read a CSV file from the web into a data frame:

- A. Use R code to read directly from a URL on the web. Store the dataset into a new dataframe, called `df_Comps`. The URL is: “<https://intro-datascience.s3.us-east-2.amazonaws.com/companies1.csv>” **Hint:** use `read_csv()`, not `read.csv()`. This is from the **tidyverse** package. Check the help to compare them.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.3      v readr      2.1.4
## v forcats    1.0.0      v stringr    1.5.0
## v ggplot2    3.4.4      v tibble     3.2.1
## v lubridate  1.9.3      v tidyr      1.3.0
```

```
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()   masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

url<-"https://intro-datascience.s3.us-east-2.amazonaws.com/companies1.csv"
df.comps<-read_csv(url)

## Rows: 47758 Columns: 18
## -- Column specification -----
## Delimiter: ","
## chr (16): permalink, name, homepage_url, category_list, market, funding_tota...
## dbl (2): funding_rounds, founded_year
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Part 2: Create a new data frame that only contains companies with a homepage URL:

- E. Use **subsetting** to create a new dataframe that contains only the companies with homepage URLs (store that dataframe in **urlComps**).

```
url.comps<-df.comps %>% filter(!is.na(df.comps$homepage_url))
```

- D. How many companies are missing a homepage URL?

```
url.NA.count<-sum(is.na(df.comps$homepage_url))
url.NA.count
```

```
## [1] 3323
```

There are 3,323 companies that are missing a homepage URL.

Part 3: Analyze the numeric variables in the dataframe.

- G. How many **numeric variables** does the dataframe have? You can figure that out by looking at the output of **str(urlComps)**.

```
glimpse(url.comps)

## Rows: 44,435
## Columns: 18
## $ permalink      <chr> "/organization/waywire", "/organization/tv-communicat~
## $ name           <chr> "#waywire", "&TV Communications", "'Rock' Your Paper~
## $ homepage_url   <chr> "http://www.waywire.com", "http://enjoyandtv.com", "~
## $ category_list  <chr> "|Entertainment|Politics|Social Media|News|", "|Game~
```

```
## $ market      <chr> "News", "Games", "Publishing", "Electronics", "Softw~
## $ funding_total_usd <chr> "1 750 000", "4 000 000", "40 000", "1 500 000", "1 ~
## $ status      <chr> "acquired", "operating", "operating", "operating", "~
## $ country_code <chr> "USA", "USA", "EST", "GBR", "USA", "USA", "ARG", NA,~
## $ state_code   <chr> "NY", "CA", NA, NA, "NY", "FL", NA, NA, "IL", NA, "C~
## $ region      <chr> "New York City", "Los Angeles", "Tallinn", "London",~
## $ city         <chr> "New York", "Los Angeles", "Tallinn", "London", "New~
## $ funding_rounds <dbl> 1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 4, 1, 1, 1~
## $ founded_at   <chr> "1/6/12", NA, "26/10/2012", "1/4/11", "1/1/12", "10/~
## $ founded_month <chr> "2012-06", NA, "2012-10", "2011-04", "2012-01", "201~
## $ founded_quarter <chr> "2012-Q2", NA, "2012-Q4", "2011-Q2", "2012-Q1", "201~
## $ founded_year  <dbl> 2012, NA, 2012, 2011, 2012, 2011, NA, 2007, 2010, NA~
## $ first_funding_at <chr> "30/06/2012", "4/6/10", "9/8/12", "1/4/11", "29/08/2~
## $ last_funding_at <chr> "30/06/2012", "23/09/2010", "9/8/12", "1/4/11", "4/9~
```

Used glimpse here instead because the output for str() was very messy.

From this output there are only two variables that are being flagged as numeric variables, although looking at the data it is evident that there should be at least one more (`funding_total_usd`) variable that should be flagged as numeric.

H. What is the average number of funding rounds for the companies in `urlComps`?

```
average.num.funding.rnds<-mean(url.comps$funding_rounds)
average.num.funding.rnds
```

```
## [1] 1.725194
```

The average number of funding rounds for the companies in `urlComps` is 1.73.

I. What year was the oldest company in the dataframe founded? **Hint:** If you get a value of “NA,” most likely there are missing values in this variable which preclude R from properly calculating the min & max values. You can ignore NAs with basic math calculations. For example, instead of running `mean(urlComps$founded_year)`, something like this will work for determining the average (note that this question needs to use a different function than ‘mean’).

```
#mean(urlComps$founded_year, na.rm=TRUE)
```

```
#your code goes here
```

```
summary(url.comps$founded_year)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
##      1900    2005    2009    2007    2012    2014     9255
```

```
#or using the suggested code above
```

```
min(url.comps$founded_year, na.rm=TRUE)
```

```
## [1] 1900
```

```
#either way results in obtaining the value of 1900
```

There are at least two ways to obtain the year the oldest company/companies (if founded in the same year). Either way the result is the same, the oldest company was founded in 1900.

Part 4: Use string operations to clean the data.

- K. The **permalink** variable in **urlComps** contains the name of each company but the names are currently preceded by the prefix “/organization/”. We can use `str_replace()` in tidyverse or `gsub()` to clean the values of this variable:

```
url.comps<- url.comps %>% mutate(permalink = str_replace(permalink, ".*/", ""))
```

```
# Verifying that the code above made the desired changes  
head(url.comps$permalink)
```

```
## [1] "waywire"          "tv-communications" "rock-your-paper"  
## [4] "in-touch-network" "n-plusn"           "club-domains"
```

- L. Can you identify another variable which should be numeric but is currently coded as character? Use the `as.numeric()` function to add a new variable to **urlComps** which contains the values from the char variable as numbers. Do you notice anything about the number of NA values in this new column compared to the original “char” one?

```
url.comps<- url.comps %>% mutate(funding_total_usd = as.numeric(funding_total_usd))
```

```
## Warning: There was 1 warning in 'mutate()'.  
## i In argument: 'funding_total_usd = as.numeric(funding_total_usd)'.  
## Caused by warning:  
## ! NAs introduced by coercion
```

```
head(url.comps$funding_total_usd)
```

```
## [1] NA NA NA NA NA NA
```

```
sum(is.na(url.comps$funding_total_usd))
```

```
## [1] 44407
```

```
#undoing the changes  
url.comps<-df.comps %>% filter(!is.na(df.comps$homepage_url))  
url.comps<- url.comps %>% mutate(permalink = str_replace(permalink, ".*/", ""))
```

All of the values stored in the variable are coerced into NA with this method/without further modification.

- M. To ensure the char values are converted correctly, we first need to remove the spaces between the digits in the variable. Check if this works, and explain what it is doing:

```
library(stringi)
url.comps$funding_new <- stri_replace_all_charclass(url.comps$funding_total_usd, "\\p{WHITE_SPACE}", "")
```

N. You are now ready to convert `urlComps$funding_new` to numeric using `as.numeric()`.

Calculate the average funding amount for `urlComps`. If you get “NA,” try using the `na.rm=TRUE` argument from problem I.

```
url.comps<- url.comps %>% mutate(funding_new = as.numeric(funding_new))
```

```
## Warning: There was 1 warning in ‘mutate()’.
## i In argument: ‘funding_new = as.numeric(funding_new)’.
```

Caused by warning:

```
## ! NAs introduced by coercion
```

```
sum(is.na(url.comps$funding_new))
```

```
## [1] 6969
```

```
mean(url.comps$funding_new, rm.na=TRUE)
```

```
## [1] NA
```

```
# nope, lets try something else then
url.comps<- url.comps %>% filter(!is.na(funding_new))
mean(url.comps$funding_new)
```

```
## [1] 18321551
```

```
# Ok, now it works? Oh well, here are two ways to do the same thing.
```

The average funding for `urlComps` is \$18.32 million.

Sample three unique observations from `urlComps$funding_rounds`, store the results in the vector ‘observations’

```
observations<- sample(url.comps$funding_rounds, 3, replace=FALSE)
```

Take the mean of those observations

```
mean(observations)
```

```
## [1] 1
```

Do the two steps (sampling and taking the mean) in one line of code

```
observations.mean<-mean(sample(url.comps$funding_rounds, 3, replace=FALSE))
observations.mean
```

```
## [1] 2.333333
```

Explain why the two means are (or might be) different.

The two means are different because the values being sampled vary every time the sample command is run. The samples are pulled from the same population but because we are only pulling 3 values these values will vary greatly each time.

Use the replicate() function to repeat your sampling of three observations of urlComps\$funding_rounds observations five times. The first argument to replicate() is the number of repeats you want. The second argument is the little chunk of code you want repeated.

```
replication1<-replicate(5, sample(url.comps$funding_rounds,3,replace=FALSE))
replication1
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    1    1    3    2
## [2,]    1    1    1    1    1
## [3,]    1    4    2    3    2
```

```
mean(replication1)
```

```
## [1] 1.666667
```

Rerun your replication, this time doing 20 replications and storing the output of replicate() in a variable called **values**.

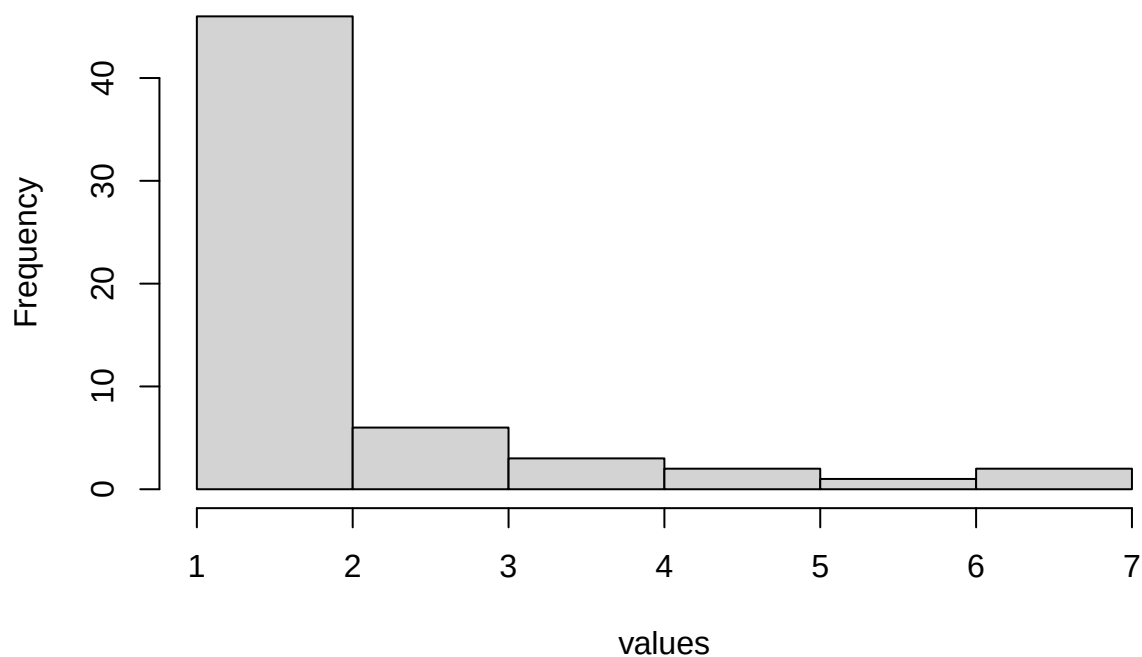
```
values<-replicate(20, sample(url.comps$funding_rounds,3,replace=FALSE))
mean(values)
```

```
## [1] 2.016667
```

Generate a **histogram** of the means stored in **values**.

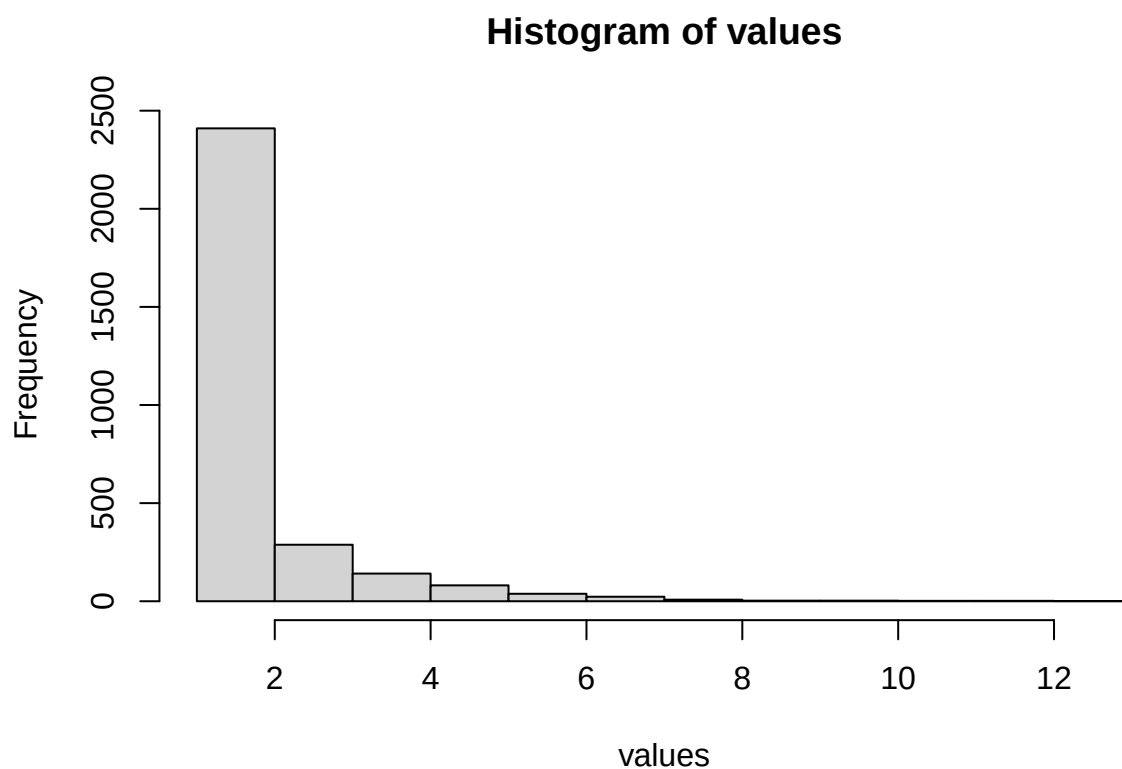
```
hist(values)
```

Histogram of values



Rerun your replication, this time doing 1000 replications and storing the output of `replicate()` in a variable called **values**, and then generate a histogram of **values**.

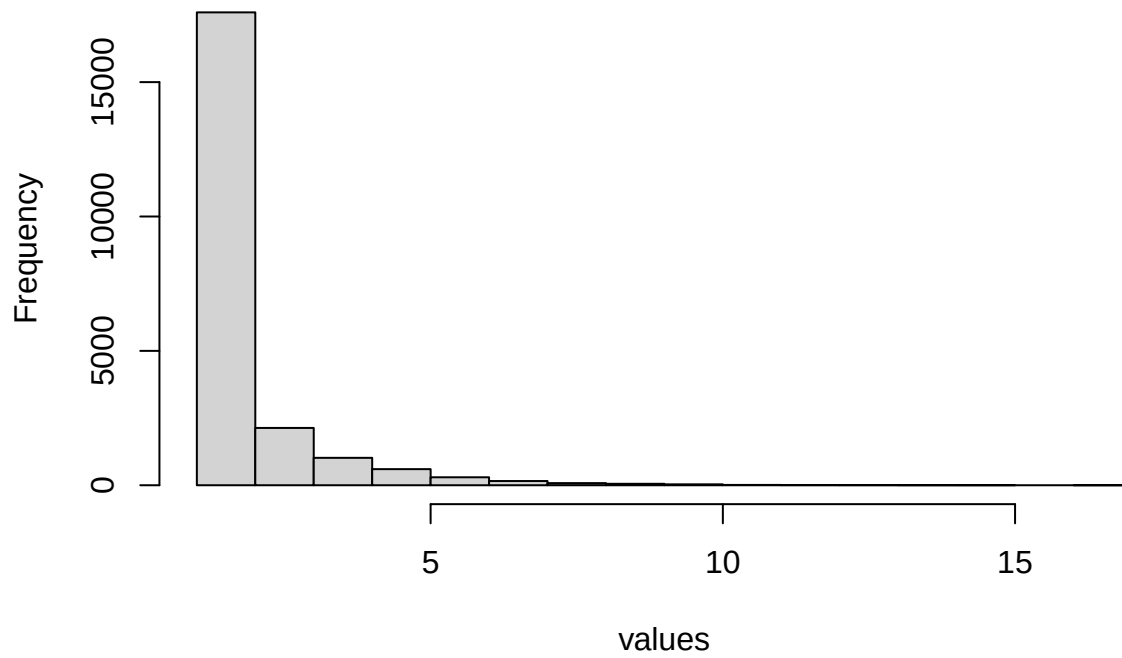
```
values<-replicate(1000, sample(url.comps$funding_rounds,3,replace=FALSE))  
hist(values)
```



Repeat the replicated sampling, but this time, raise your sample size from 3 to 22. How does that affect your histogram? Explain in a comment.

```
values<-replicate(1000, sample(url.comps$funding_rounds,22,replace=FALSE))  
hist(values)
```

Histogram of values



Explain in a comment below, the last three histograms, why do they look different?

The histograms look different because the higher the replication count or the higher the sample number